

In this new version of `get_formatted_name()`, the middle name is optional. If a middle name is passed to the function, the full name will contain a first, middle, and last name. Otherwise, the full name will consist of just a first and last name. Now the function should work for both kinds of names. To find out if the function still works for names like Janis Joplin, let's run the test again:

```
$ pytest
===== test session starts =====
--snip--
test_name_function.py . [100%]
===== 1 passed in 0.00s =====
```

The test passes now. This is ideal; it means the function works for names like Janis Joplin again, without us having to test the function manually. Fixing our function was easier because the failed test helped us identify how the new code broke existing behavior.

Adding New Tests

Now that we know `get_formatted_name()` works for simple names again, let's write a second test for people who include a middle name. We do this by adding another test function to the file `test_name_function.py`:

```
test_name
_function.py from name_function import get_formatted_name

def test_first_last_name():
    --snip--

def test_first_last_middle_name():
    """Do names like 'Wolfgang Amadeus Mozart' work?"""
❶ formatted_name = get_formatted_name(
    'wolfgang', 'mozart', 'amadeus')
❷ assert formatted_name == 'Wolfgang Amadeus Mozart'
```

We name this new function `test_first_last_middle_name()`. The function name must start with `test_` so the function runs automatically when we run **pytest**. We name the function to make it clear which behavior of `get_formatted_name()` we're testing. As a result, if the test fails, we'll know right away what kinds of names are affected.

To test the function, we call `get_formatted_name()` with a first, last, and middle name ❶, and then we make an assertion ❷ that the returned full name matches the full name (first, middle, and last) that we expect. When we run **pytest** again, both tests pass:

```
$ pytest
===== test session starts =====
--snip--
collected 2 items

❶ test_name_function.py .. [100%]
===== 2 passed in 0.01s =====
```
